



Machete de comandos — Clase 6 — Consumir un modelo de IA por API

Seminario de Actualización · Tecnicatura Superior en Ciencia de Datos e IA · IFTS N.º 18

Docente: Cynthia M. Villagra · Ciclo lectivo 2026 · Segundo cuatrimestre

Todos los comandos de la clase, en el orden en que se usan. Copiá y pegá reemplazando lo que esté <entre corchetes> por lo tuyo.

1. Instalar la librería

```
pip install huggingface_hub
```

Y agregala al requirements.txt de tus dos apps (Gradio y Streamlit):

```
huggingface_hub
```

2. El token, en la terminal (solo para esta sesión)

Generalo en huggingface.co/settings/tokens, con permiso **Inference Providers**. La sintaxis cambia según la terminal:

```
# PowerShell
$env:HF_TOKEN = "<tu token>"

# Git Bash
export HF_TOKEN="<tu token>"
```

| Si pegás la de Git Bash en PowerShell, falla. Fijate primero qué terminal tenés abierta.

3. El primer llamado al modelo

```
import os
from huggingface_hub import InferenceClient

client = InferenceClient(
    provider="hf-inference",
    api_key=os.environ["HF_TOKEN"],
)

resultado = client.text_classification(
    "esta clase me está costando pero se entiende",
    model="cardiffnlp/twitter-xlm-roberta-base-sentiment",
    top_k=3,
)

for r in resultado:
    print(r.label, r.score)
```

Qué es cada parte

En el código	Qué es
InferenceClient	la clase
client	el objeto (la instancia)

En el código	Qué es
provider, api_key	parámetros; lo que va después del = son los argumentos
text_classification	un método de client
top_k=3	pide las 3 etiquetas más probables
resultado	una lista de objetos TextClassificationOutputElement
r.label, r.score	atributos de cada elemento

4. Si algo falla

Error	Quién lo da	Qué hacer
KeyError: 'HF_TOKEN'	tu compu	No definiste la variable en esta terminal. Volvé al paso 2.
401 Unauthorized	Hugging Face	El token está mal copiado o no tiene el permiso. Generá uno nuevo.

5. Sumarlo a tu app de Gradio (Render)

Una pestaña nueva, sin tocar lo que ya andaba:

```
def analizar_sentimiento(texto):
    resultado = client.text_classification(
        texto,
        model="cardiffnlp/twitter-xlm-roberta-base-sentiment",
        top_k=3,
    )
    return {r.label: r.score for r in resultado}

with demo:
    with gr.Tab("Análisis de sentimiento"):
        entrada = gr.Textbox(label="Escribí un texto")
        boton = gr.Button("Analizar")
        salida = gr.Label(label="Resultado")
        boton.click(analizar_sentimiento, inputs=entrada, outputs=salida)
```

En Render: panel de tu servicio → **Environment** → variable nueva HF_TOKEN con tu token.

6. Lo mismo en Streamlit (Community Cloud)

```
import os
import streamlit as st
from huggingface_hub import InferenceClient

@st.cache_resource
def crear_cliente():
    return InferenceClient(provider="hf-inference",
                           api_key=os.environ["HF_TOKEN"])

client = crear_cliente()
texto = st.text_input("Escribí un texto")
```

```

if st.button("Analizar") and texto:
    for r in client.text_classification(
        texto,
        model="cardiffnlp/twitter-xlm-roberta-base-sentiment",
        top_k=3,
    ):
        st.write(r.label, round(r.score, 2))

```

| @st.cache_resource hace que el cliente se cree una sola vez. Sin eso, Streamlit crea uno nuevo en cada clic.

En Streamlit Community Cloud: tu app → **Settings** → **Secrets**:

```
HF_TOKEN = "<tu token>"
```

Los secrets también quedan como variables de entorno, así que `os.environ["HF_TOKEN"]` funciona igual. Para probar local, el archivo `.streamlit/secrets.toml`, **agregado al `.gitignore`**.

7. Documentar en el README

En la carpeta Clase_06 de tu repositorio, un README.md con esta estructura (completá lo tuyo):

```

# Clase 06 – Consumir un modelo de IA por API

Modelo: cardiffnlp/twitter-xlm-roberta-base-sentiment (análisis de sentimiento)

- App en Render: <link>
- App en Streamlit: <link>

El token (HF_TOKEN) está configurado en Environment (Render) y en Secrets (Streamlit).
No está en el código ni en el repositorio.

```

Después, en el README general de la raíz, sumá la Clase 06 al índice y al estado del proyecto.

| Nunca pegues el token en el README: el repositorio es público.

8. Revisión de pares: clonar el repo de un compañero

```

cd C:\repos
git clone <link del repo de tu compañero>

```

| Solo lectura: nunca `git push` sobre el repo ajeno. Si Windows dice "Filename too long", cloná más cerca de la raíz (`C:\repos\`).